Deployment, Monitoring, Maintenance, and Retrospective

COMP-7431: Advanced Software Engineering | Week 12
CampusConnect: make the MVP operable—and learn from what happens

Release responsibly. Observe reality. Escalate safely. Improve with evidence.

Monday Morning: Red Team Findings Become Operating Rules

Week 11 exposed prompt injection, poisoned documents, leakage, unsafe actions, and synthetic-media trust risks.
Week 12 converts selected findings into workflow controls, alerts, escalation, and maintenance evidence.

WHAT THE TEAM BRINGS FORWARD

"CampusConnect refuses unsupported answers, cites approved evidence, preserves text fallback, and records its highest residual risk."

WHAT OPERATIONS MUST NOW ANSWER

Who sees an unsupported case? What is logged? When does a human intervene? How is a failed change reversed? What evidence shapes the next release?

A risk record is useful only when the released service changes what people do when the risk appears.

A Working MVP Is Not Yet an Operable Service

WORKING MVP

A prepared demonstration succeeds when the expected components and data are available.

Example: A known Wi-Fi question retrieves the right passage, produces a citation, and plays optional voice.

OPERABLE SERVICE

People can detect failure, route exceptions, understand impact, restore service, and learn without inventing a response during the incident.

Example: An unsupported question creates a review row with time, question, reason, owner, status, and resolution.

RELEASE CONFIDENCE

The team has a bounded release, named owners, observable signals, an escalation path, a maintenance cadence, and a tested rollback.

Example: A bad prompt change triggers rollback to the tagged release candidate and a documented retest.

Deployment makes software available. Operations keeps it trustworthy after availability.

Week 12 Completes the Software Delivery Loop

RELEASE WITH CONTROL

Define the release boundary, dependencies, owner, change record, and rollback point before exposing the service.

Example: Release the tested Gradio MVP; do not silently add an untested action connector.

OPERATE WITH EVIDENCE

Monitor meaningful events, route unsupported cases, escalate impact, and preserve a minimal diagnostic trail.

Example: A reviewer labels the case approved, corrected, or rejected—and records why.

LEARN AND ITERATE — Use incidents, review outcomes, user friction, and residual risk to choose the next change—for example, convert a repeated knowledge gap into one prioritized backlog item.

Operational Responsibilities Need Named Owners

RELEASE

Approve exactly what version, configuration, dependencies, and data become available.

EXAMPLE
Tag the tested commit and record the prompt and knowledge version.

OBSERVE

Collect meaningful user and system signals without storing unnecessary personal data.

EXAMPLE
Count unsupported cases; never copy passwords into the review sheet.

RESPOND + RECOVER

Route exceptions, escalate impact, apply a safe workaround, and restore a known-good version.

EXAMPLE
Disable voice while text remains healthy; roll back a harmful prompt.

LEARN: Repeated Wi-Fi abstentions become a knowledge-base improvement—not a lower abstention threshold.

Operate as a Closed Learning Loop

1 RELEASE

Expose a bounded, tagged version.

Example: v0.9.0-rc1 plus approved prompt and knowledge files.

2 OBSERVE

Capture meaningful behavior and failures.

Example: supported, unsupported, error, latency, fallback used.

3 RESPOND

Route, escalate, contain, or recover.

Example: send unsupported cases to review; preserve text service.

4 LEARN

Prioritize the next change from evidence.

Example: repair a missing source, add a regression case, revise the runbook.

A retrospective closes the loop only when evidence changes the backlog, architecture, or operating rule.

The AI Periodic Table Reaches AI Operations

APPLIED AI
SYSTEMS — X

Release the whole user journey—not merely a model endpoint.

CampusConnect includes RAG, citations, voice, video, fallback, and human review.

AI OPERATIONS —
XI

Observe quality, latency, failures, review outcomes, data changes, and rollback readiness.

Operational evidence reveals whether the system remains useful after launch.

GUARDRAILS — VI

Turn residual risks into routing, escalation, confirmation, retention, and permission controls.

Human review is a designed boundary—not an apology for imperfect AI.

AUTOMATION
PLATFORMS — IX

Use n8n to coordinate events and systems while preserving ownership and limits.

Automation moves evidence; it does not decide truth or grant authority by itself.

Week 12 works mainly in AI Operations while connecting Guardrails, Automation Platforms, and the entire applied system.

Define the Workflow Contract Before Drawing Nodes

WORKFLOW CONTRACT

WHEN [event] occurs, ROUTE [minimum data] to [owner], RECORD [outcome], ESCALATE when [condition], and RECOVER by [safe action].

- Trigger: unsupported answer, system error, slow response, failed dependency, or reviewer rejection
- Minimum payload: case_id, timestamp, question summary, support status, reason, source references, severity
- Owner: named reviewer or on-call role—not “someone”
- Outcome: approved response, corrected knowledge, rejected request, duplicate, or incident
- Privacy boundary: no passwords, secret keys, private records, or unnecessary identifiers

A workflow is an operational promise. If the trigger, owner, data, and completion state are vague, the diagram is only decoration.

CampusConnect Separates Answering from Exception Routing

ANSWERING PATH

RAG retrieves approved evidence; the application returns answer_text, source references, and support_status.

Example: support_status = supported; the normal user journey continues.

EXCEPTION EVENT

The application or lab payload describes the unsupported condition in a small, structured record.

Example: case_id, question, reason = no approved evidence, severity = normal.

HUMAN REVIEW PATH

n8n routes the event to a Google Sheet or reviewer queue and records status and resolution.

Example: reviewer adds "correct knowledge gap; create issue KB-14."

MAINTENANCE PATH — GitHub receives the durable engineering work: source repair, prompt change, test, owner, and release decision. The workflow coordinates the exception; it does not fabricate the missing answer.

Route an Unsupported Answer to Human Review

1 Detect unsupported → 2 Webhook receives → 3 IF routes → 4 Sheet logs → 5 Human reviews → 6 Issue or close

CAMPUSCONNECT EXAMPLE

"Can graduate assistants waive parking fines?" No approved source supports it. The sheet records OPEN; a reviewer marks REJECTED—authority unavailable—and links a scope issue.

Use the same case_id from detection through closure so the team can trace the outcome.

Confirmed abstention can be the safe result. The workflow must not fabricate the missing answer.

Each n8n Node Has One Bounded Responsibility

Node	CampusConnect responsibility	Example evidence
Webhook	Receive a structured exception event	Timestamp + case_id + support_status
IF	Apply a visible deterministic route	unsupported → review; supported → no review
Google Sheets	Create or update the review record	OPEN → REVIEWED → CLOSED
HTTP Request	Call an approved external endpoint when needed	Send a notification—not a secret or autonomous action

A good workflow makes conditions, data movement, ownership, and failure visible. A long canvas is not evidence of better architecture.

Monitoring Starts with Questions, Not Dashboards

MONITORING QUESTIONS

1. *Can users complete the task safely?*
2. *Are dependencies and latency healthy?*
3. *Are grounding and abstention changing?*
4. *Are alerts owned and recovery materials usable?*

USER OUTCOME

Supported-answer rate, fallback success, repeated unresolved topic.

SYSTEM HEALTH

Webhook failure, sheet-write error, slow RAG response, voice unavailable.

AI QUALITY

Grounding, citations, abstention, and human-review outcomes over time.

CONTROL HEALTH

High-severity cases assigned; rollback target and runbook remain usable.

A metric earns its place only when a person knows what decision or action follows from it.

Escalation Matches Impact, Not Technical Drama

NORMAL REVIEW

Unsupported but safe request; route to the review queue.

Example: missing cafeteria-hours source.

ELEVATED

Repeated failure, misleading official guidance, inaccessible fallback, or material workflow outage.

Example: every Wi-Fi question routes unsupported after a document update.

CRITICAL

Sensitive-data exposure, unsafe authorized action, credible security incident, or widespread harmful advice.

Example: logs contain a real credential—stop processing, preserve evidence safely, and notify the responsible authority.

ESCALATION RECORD — Record severity, impact, time, owner, containment, communication, resolution, and residual risk. Severity follows consequence and reach—not technical drama.

Maintenance Preserves Reliability After the Demo

KNOWLEDGE

Review source ownership, freshness, gaps, and poisoning controls.

EXAMPLE
Retire an outdated Wi-Fi document and rerun retrieval cases.

PROMPTS + RULES

Version every change; retest citations, abstention, and injection defenses.

EXAMPLE
Do not weaken abstention merely to improve answer rate.

Maintenance is planned work with an owner and cadence—not a promise to "check it later."

Rollback Restores a Known-Good State

ROLLBACK TRIGGER

Define the condition that makes continued operation riskier than reversal.

EXAMPLE
A prompt update breaks abstention or the review workflow loses cases.

KNOWN-GOOD TARGET

Name the exact tagged commit, configuration, prompt, knowledge snapshot, and workflow version.

EXAMPLE
Restore v0.9.0-rc1 plus the last tested n8n workflow export.

RECOVERY STEPS

Disable the bad path, restore the target, verify critical checks, communicate status, and preserve incident evidence.

EXAMPLE
Keep text available while voice is disabled; retest supported and unsupported questions.

FORWARD REPAIR — Fix on a new branch, repeat the gate, and release deliberately. Rollback is an engineered way to reduce harm while the team learns.

Practice One Failure Before Claiming Readiness

1 CHOOSE — Select one safe, reversible failure.

Example: send support_status = unsupported or temporarily use an invalid sheet tab.

2 OBSERVE

Identify what signal appears and where evidence is stored.

Example: n8n execution fails; the case is absent from the review row.

3 RESPOND

Follow the runbook: contain, assign, correct, and communicate.

Example: repair the tab name and rerun the same case_id.

4 VERIFY

Confirm recovery and check for duplication or lost data.

Example: exactly one completed review row exists.

5 IMPROVE — Update monitoring, instructions, or rollback evidence; add a sheet-write failure check to the runbook.

A recovery plan becomes credible when the team rehearses a bounded failure and records what actually happened.

The Retrospective Converts Evidence into the Next Iteration

Evidence	Interpretation question	Next action example
Unsupported reviews	Which gaps repeat—and which should remain out of scope?	Add one approved source; clarify one refusal
Failures + latency	Where did the journey become unreliable?	Preserve text fallback; repair sheet-write handling
Red Team findings	Which residual risk changed after controls?	Add regression case; narrow permission
Team process	What slowed review, ownership, or recovery?	Name owner; simplify handoff; update runbook

A retrospective is not a feelings-only closing ritual. It combines team experience with release, testing, Red Team, workflow, and user evidence.

Prepare Your Own MVP and Seven-Minute Engineering Story

YOUR TEAM'S MVP

Choose a subject other than CampusConnect. Build a chatbot with the artifacts that matter: grounded knowledge/RAG, voice, video, fallback, testing, Red Team evidence, and human review.

You do not need every feature merely because CampusConnect used it. Keep only the features that support a clear user need and can be demonstrated reliably.

SEVEN-MINUTE GROUP PRESENTATION

Student 1 — PowerPoint story: problem, users, architecture, design decisions, evidence, risks, operations, and next iteration.

Student 2 — Live demo: one supported journey, one multimodal feature, one unsupported or failure path, and the human-review outcome.

FINAL-PROJECT SIMPLIFICATION

You may omit the semester's detailed GitHub activity from the presentation. Focus the seven minutes on the working MVP and the engineering reasoning behind it.

BEFORE PRESENTATION DAY

CampusConnect was the rehearsal. Rehearse your handoff, prepare a screenshot/video fallback, remove credentials and personal data, and make the demo readable and audible.